

The Automated Commercial Micro Climate Nursery

COMP50069 Hardware Microcontrollers and Sensors | Scenario 2

Ismail Azaan | CB012442

Submission date: Sept 25, 2026

Wokwi simulation: <https://wokwi.com/projects/476106928243893249>

Git repository: <https://github.com/howdynet22/ESP32-Nursery-Scenario-2>

This report explains a small ESP32 system for a nursery. It reads indoor temperature, humidity and light, then adjusts a vent and two LEDs that stand in for grow lights. A switch selects Manual mode, where a worker can set the vent and lights with two buttons. If a sensor gives an invalid reading, the system enters Fault mode and waits for a deliberate reset. The supplied code and circuit files support a Wokwi simulation.

Recorded simulation results and pending hardware checks are in Section 6.

1 Scenario and Scope

Seedlings can be harmed by excess heat or low light. This ESP32 prototype reads indoor temperature, humidity and brightness, moves a model vent, and switches two LEDs that represent grow lights. The screen and serial monitor show its state. A slide switch selects Manual mode, two buttons control the vent and lights, and another button resets a repaired fault. The assignment requires a physical demonstration; Wokwi provides extra evidence.

This is a model of one growing area. It displays humidity but has no humidifier, and its LEDs stand in for real grow lamps. The example limits are 29 °C for opening the vent, 17 °C for closing it against indoor cold, and 30% relative brightness for turning the LEDs on. A real nursery would need appropriate lamps, safe wiring and limits chosen for its crop.

MoSCoW prioritisation

Priority	Requirements and rationale
Must have	ESP32; DHT22 and LDR; servo, LEDs and I2C OLED; Auto, Manual and Fault modes; light measurement (ADC), servo signals (PWM), serial commands (UART), and regular timed checks. These parts and behaviours are required by the brief. Together they let the nursery measure its conditions, move the vent, show when lights should turn on, and respond to a broken sensor.
Should have	Hysteresis prevents rapid switching near a limit: lights turn on at 30% brightness and off at 36%. After a fault, both sensors must work for six seconds before a worker presses reset. Allowed ranges for settings prevent unsuitable inputs. These choices make control steadier and recovery deliberate.
Could have	Future versions could add an outdoor sensor, real lamps, a humidifier and a second sensor for checking readings. These need extra hardware and tests. An outdoor sensor, for example, would help prevent opening the vent into cold air.
Will not have	This version will not predict weather, use cloud access, or control multiple growing areas. It cannot sense outdoor weather or confirm that the vent moved. For those reasons, a person must supervise it.

The controller checks problems in this order: first a sensor fault, then the maintenance switch, then normal automatic control. A broken sensor therefore cannot be hidden by switching to Manual mode. When both sensors are healthy, the worker can use Manual mode to set the vent to 0°, 45° or 90° and switch the LEDs on or off.

In Fault mode, the system sets the vent to 45° and turns both LEDs on. This is a chosen middle position while the temperature is unknown; it cannot be guaranteed safe for every crop. The fault stays active until both sensors give valid readings for six continuous seconds and a worker presses reset. The first valid reading starts the timer, and any later invalid reading starts the wait again. A reset button press is accepted only after its contact has been stable for 40 ms. This prevents a brief normal reading or contact bounce from restarting the system. The model's middle vent position may release some heat but could also admit cold air; keeping the LEDs on guards against darkness but may add heat. The controller has no outdoor sensor or vent-position feedback, so a worker must supervise it. A real installation would need outdoor measurements, independent limits and physical safeguards for power loss or a jammed vent.

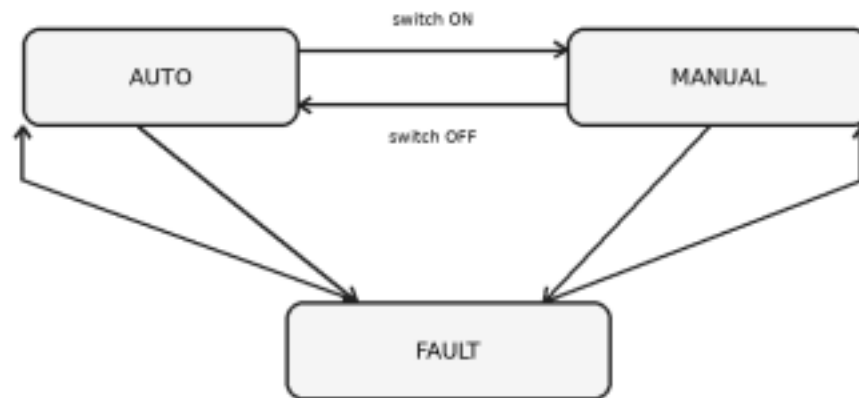
The design treats a servo angle of 0° as a closed vent and 90° as a fully open vent. On a real vent, those angles depend on how the parts are connected. The installer must check that the servo cannot push beyond the vent's mechanical limits. Humidity stays on the display so the worker can see it, but the controller cannot correct it. Settings can be changed through the serial monitor while the system runs; they return to their defaults after a restart because they are not saved.

2 System Design

The DHT22 sends digital temperature and humidity readings to the ESP32. The LDR sends a changing voltage that the ADC turns into a number; ADC means analogue-to-digital converter. The slide switch and three pushbuttons are digital inputs. The ESP32 sends on/off signals to two LEDs, PWM pulses to position the servo, and information to the OLED and USB serial monitor. PWM means pulse-width modulation. GPIO34 reads the LDR, GPIO21 and GPIO22 connect the OLED, and GPIO25 controls the servo. The main loop checks new readings and updates outputs without a long intentional pause.

Current state	Condition	Next state and action
Auto or Manual	DHT22 reading invalid or light reading at its lowest or highest possible value	Enter Fault and stay there until reset; vent 45°, LEDs on, show the sensor problem.
Auto	Manual switch unchanged for 40 ms	Manual; vent 0° and LEDs off on entry; buttons set vent to 0°/45°/90° and toggle LEDs.
Manual	Manual switch moved back and unchanged for 40 ms	Auto; temperature and light regain control.
Fault	A sensor is still bad, or both have been healthy for less than 6 s	Remain Fault; reset press is ignored.
Fault	Both sensors healthy for at least 6 s, then worker presses reset	Return to Manual or Auto according to switch position.

The state diagram shows what makes the controller move between Auto, Manual and Fault. A fault always has priority, even if the worker moves the switch at the same time.



Diagonal arrows into FAULT: invalid DHT22 or LDR.
 Outer arrows out of FAULT: both valid for 6 s + new reset press.
 Destination after reset follows the current manual switch.

On every pass through the main loop, the controller checks for a fault before checking the Manual switch. If a fault is active, changing the switch does not change the vent or LEDs. Its position still determines which mode the system returns to later. Any bad sensor reading starts the six-second recovery wait again.

To leave Fault mode, the worker must wait until both sensors have stayed healthy for six seconds, then press and release reset. Holding the button down early does not count as a new press. The serial reset command follows the same rule. Just after power-up, before the first DHT22 reading, the screen shows N/A and the vent waits at 45°. This avoids treating the initial absence of a reading as a permanent fault.

The scenario mentions the danger of opening a vent when it is cold outside. This build has no outdoor sensor, so it cannot detect that danger. It can only close the vent when the indoor temperature falls to 17 °C or below. The vent starts opening when the room reaches the hot setting, normally 29 °C, and opens farther as the room gets hotter, reaching 90° near 38 °C. It does not immediately close when the temperature drops slightly; that gap is hysteresis and prevents repeated movement. A real installation would need an outdoor sensor and a safe weather rule. Fault mode never uses a bad reading to calculate vent position.

3 Feature Implementation

The Manual switch is connected to GPIO27; the reset, vent and light buttons use GPIO26, GPIO32 and GPIO33. The LEDs use GPIO18 and GPIO19. The switch and buttons use the ESP32's built-in pull-up resistors, so connecting an input to ground makes it active. The program waits for each switch or button to stay in its new position for 40 ms before accepting it; this filters contact bounce. Each LED has its own 220 Ω resistor to limit current. Arduino's digitalRead and digitalWrite keep the code readable while using the ESP32's digital pins. The DHT22 is read about every two seconds, as required by the sensor library.

The LDR module changes its output voltage with light level. The ESP32 converts that voltage to a number from 0 to 4095 on GPIO34. This module's number rises in darkness, so the code reverses it to show a relative brightness percentage: $(4095 - \text{reading}) \times 100 / 4095$. The percentage is useful for this demonstration, but it is not a calibrated measurement in lux. A reading at the very bottom or top of the ADC range is treated as a possible wiring or sensor fault. That check will not catch every disconnection, so a real system would need stronger checks.

The LEDs turn on when relative brightness reaches 30% or less. They stay on until it reaches 36% or more, which prevents flickering around one cut-off point. Auto keeps its own light decision while Manual is active, so returning to Auto uses the sensor state rather than the last manual setting. The program also rejects temperature or humidity readings that are missing or outside the DHT22's stated ranges. In Auto mode, the servo moves through intermediate angles as the room warms. In Manual, the vent button cycles 0°, 45° and 90°, and the light button toggles both LEDs. Fault mode commands 45°. The servo uses PWM control at 50 Hz. The serial command set hot accepts only 25–34 °C so the temperature calculation remains valid.

The program uses `millis()`, the ESP32's elapsed-time counter, to schedule LDR and screen updates every 250 ms, DHT22 reads every 2000 ms, and UART status every 1000 ms. The main loop contains no `delay()`, so it continues to check inputs between scheduled tasks. A falling-edge interrupt on GPIO26 records each reset-button event; the loop then applies a 40 ms debounce check before acting. The interrupt routine only sets a flag and never reads sensors or drives outputs. The six-second healthy-sensor recovery wait also uses elapsed time. The OLED uses I2C on GPIO21 and GPIO22 at address 0x3C. It shows readings, vent angle, light state and faults; an invalid value appears as N/A.

The USB serial monitor runs at 115200 baud. Type `help` to see commands or status to inspect the system. `set hot` and `set dark` change the temperature and light limits until the next restart. `fault dht on/off` and `fault ldr on/off` deliberately simulate sensor problems so you can demonstrate the safety behaviour. `reset` only clears a fault after the normal recovery conditions are met. Commands are read into a fixed 64-byte space and checked for excessive length. Before real use, the fault-simulation commands would need to be removed or restricted.

Here, nonblocking means the program avoids long planned waits, so it can respond to the switch while scheduled tasks are running. A sensor read or screen update still takes a little time; nonblocking does not mean every action is instant. The program spreads reads and screen updates over time and handles serial input only when bytes are available. It also compares elapsed times in a way that continues to work when the `millis()` counter wraps around. You still need to measure the real time from an input change to an output change on the physical build.

The program can tell the servo which angle to move to, but it has no sensor to confirm that the vent actually moved. It can switch an LED pin on, but it cannot confirm that a real grow lamp is shining. If the OLED fails to start, the USB serial monitor can still show messages, provided that connection is available. A commercial version would need ways to detect a stuck vent, failed lamp and power loss, plus an outdoor temperature sensor and an independent overheating safeguard.

These are hardware changes with their own wiring and tests. They cannot be added safely by changing only the program.

4 User Manual

To recreate the VS Code simulation, clone the Git repository named on the cover and open its root folder in VS Code. Install PlatformIO IDE and Wokwi Simulator, run PlatformIO Build, then run Wokwi: Start Simulator. Activate a Wokwi licence if prompted. Keep the simulator tab visible and wait about two seconds for the first DHT22 reading. Change the DHT22 temperature and the LDR light setting to see Auto respond. The Wokwi LDR slider is labelled in lux, while the OLED shows a relative brightness percentage; use the OLED percentage to judge the 30% LED-on and 36% LED-off thresholds. Move the slide switch right for Manual: the vent starts at 0° and LEDs start off. Press the vent button to cycle 0° → 45° → 90° → 0°; press the light button to toggle the LEDs. The OLED shows MANUAL OVERRIDE and VENT/LIGHT BUTTONS. Move the switch left to return to Auto. For fault testing, open a separate PowerShell terminal in the repository folder. Install PySerial with `python -m pip install pyserial` if Python reports that

serial is missing. Restart Wokwi after changing wokwi.toml, which forwards its UART to port 4000. Run python send_command.py fault dht on. Check for SENSOR FAULT, vent 45°, and LEDs on. Run python send_command.py fault dht off, wait six healthy seconds, then run python send_command.py reset. Use the Wokwi Terminal for live output and PowerShell for command input. On the Wokwi website, enter commands in its serial monitor. Save a website copy and put its share URL on the cover.

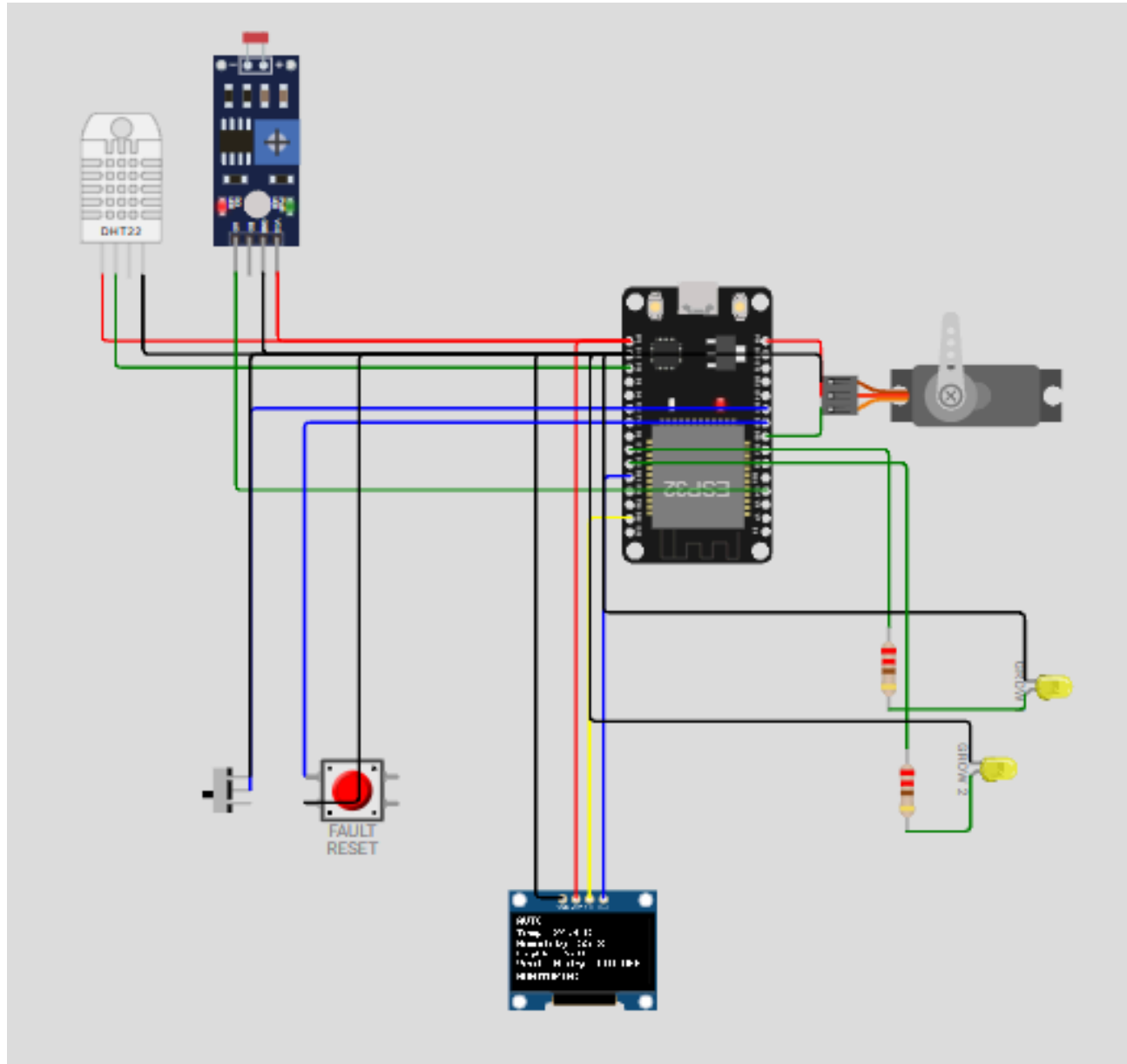
Part	ESP32 connection	Supply or physical note
DHT22	DATA GPIO15	3.3 V, common ground; use pull-up if your breakout lacks one.
LDR module	AO GPIO34	3.3 V only, common ground; never send 5 V to ADC.
I2C OLED	SDA GPIO21; SCL GPIO22	3.3 V; default 0x3C, check your module.
Servo	PWM GPIO25	External regulated 5 V and common ground for physical build.
Two LEDs	GPIO18 and GPIO19	Each through a separate 220 ohm resistor to anode; cathode to ground.
Manual switch	Common GPIO27; right contact ground	INPUT_PULLUP; left contact unused.
Reset, vent and light buttons	GPIO26, GPIO32 and GPIO33 to ground	Normally open INPUT_PULLUP; each button has 40 ms debounce.

For the physical build, turn off power before connecting parts. Check the ESP32 pin labels against the wiring table. The servo needs a suitable regulated 5 V supply and a ground shared with the ESP32. Fit the vent linkage so 0° closes it and 90° opens it without hitting a mechanical stop. Upload the sketch in Arduino IDE or PlatformIO, check the OLED address, and open the board's USB serial monitor at 115200 baud. The Python helper above connects only to Wokwi; on physical hardware, type the same fault commands into the board's serial monitor. Never connect a grow lamp directly to an ESP32 pin. To demonstrate Fault without unplugging a live wire, enter fault dht on, observe the warning and 45° vent, enter fault dht off, wait six healthy seconds, then press the reset button or enter reset. If Manual is selected, the system returns to Manual.

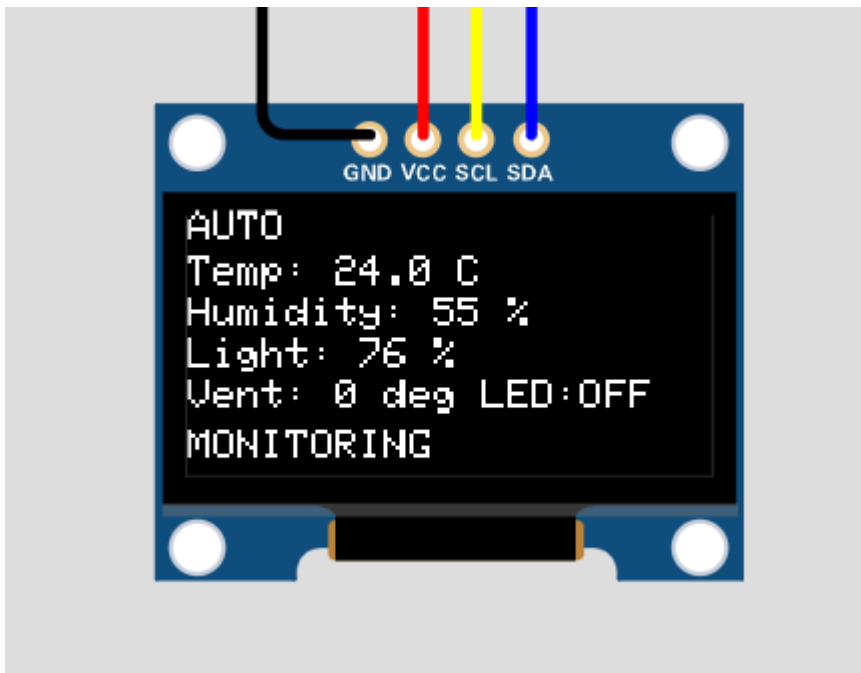
Start with the switch in Auto and the DHT22 at 24 °C; after the first reading, the OLED should show a closed vent at 0°. In Wokwi the arm points upward at this angle; the arm alone is not a vent model. Raise temperature to 29, 33 and 38 °C, waiting at least two seconds after each change. The OLED vent angle and servo arm should increase, reaching 90° at 38 °C. Lower temperature to 27 °C to show hysteresis. Adjust the LDR while watching the OLED brightness percentage: LEDs turn on at or below 30% and off at or above 36%. Move to Manual; confirm vent 0° and LEDs off, then use the vent button for 45° and 90° and the light button for on and off. Hold Manual at 90° to show the maintenance vent-open setting. Return to Auto and check that sensors control both outputs. Changing humidity changes the display but not the vent. A fault cannot be reset until both sensors are healthy for six seconds.

Wokwi circuit and live display

ESP32 nursery simulator showing the DHT22, photoresistor, servo, OLED, fault reset, and output LEDs.



Evidence below is grouped by test ID. Screen images are cropped to the simulator; results and remaining tests are recorded in the testing table.



5 Commercial IoT Appraisal

Bluelab/Autogrow IntelliClimate is a commercial controller for a growing room. Its manufacturer says it measures temperature, humidity and light and can control equipment such as lights, fans, heaters and humidifiers. It can also use an optional outdoor temperature sensor and growth-stage schedules [1]. Our prototype also reads indoor conditions and makes lighting decisions, but it moves only one example vent and uses small LEDs instead of real lamps. It has no heater, outdoor sensor or growth-stage schedule. The comparison shows what extra equipment a practical nursery system needs.

IntelliClimate is the closer comparison because it measures the same three indoor conditions shown by this prototype. The important difference is that it can control real climate equipment, while our LEDs only show the intended on/off decision. To add real equipment, we would need suitable electrical switching and a way to check that the equipment responded. Our Manual switch and two control buttons provide simple local maintenance controls. The cited product page mentions safety features but does not describe exactly what it does when a sensor fails, so we cannot claim its fault recovery works like ours.

Priva Connex is a commercial greenhouse control system. Priva says it brings together lighting, climate, water and energy management and allows remote access [2]. Its vent guidance also takes wind into account [3]. Our prototype opens one vent farther when the indoor room gets hotter, but it cannot measure wind or coordinate several vents. The key lesson is that a controller can only make decisions from information it measures. One indoor DHT22 and one LDR cannot reveal outdoor cold, wind, CO2 levels or energy use.

Connex also shows why our LDR percentage is only a simple teaching measure. Commercial lighting decisions consider the amount and type of light reaching plants. Our LDR gives a rough relative brightness value. A real lighting plan would need a calibrated sensor near the plants and measurements of the actual lamps. Priva considers wind because the same vent angle can produce different airflow in different weather.

Our program assumes that opening the vent farther will generally increase ventilation, but it does not measure

airflow or guarantee a particular temperature change. This is why the prototype needs supervision and more sensors before it could be used as a commercial controller.

Dimension	This prototype	IntelliClimate [1]	Priva Connex [2, 3]
Inputs	Indoor temperature and humidity; relative light	Temperature, RH, light; optional outside sensor	Integrated greenhouse process data
What it controls	One vent servo, two LEDs	Fans, lights, heaters, humidity and CO2 equipment	Vent, lighting, water and energy processes
Supervision	Local screen, serial monitor, switch, manual buttons and reset	PC connection and scheduling	Remote operator and management software
Scale	Single teaching enclosure	Grow-room environment	Commercial greenhouse management

6 Testing

The Wokwi results below use observed OLED, servo, LED and UART output. Pending means that the stated condition has not been fully demonstrated. The physical sensor-disconnection test still needs hardware. Wokwi's LDR slider is in lux; the controller's 30% ON and 36% OFF thresholds use relative brightness shown on the OLED. Record any discrepancy and repeat the test after fixing it.

ID and input	Expected result	Actual result	Status
T01 24 °C, bright light, switch Auto	Auto; vent 0°, LEDs off; readings shown	Observed: Auto, 24.0 C, light 99%, vent 0 deg, LEDs off. See T01 image.	Pass
T02 raise DHT to 33 °C	Auto; vent angle between 0° and 90°	Observed: Auto at 33.1 C commanded vent 50 deg; at 34.1 C, 58 deg. See T02 image.	Pass
T03 raise to 38 °C	Auto; vent 90°	Observed: Auto at 38.4 C commanded vent 90 deg. See T03 image.	Pass
T04 lower to 27 °C then 17 °C	Vent closes; cold guard applies	Observed: 33.1 C vent open; 26.7 C and 17.1 C vent 0 deg. See T04 image.	Pass
T05 darken LDR below 30%, then brighten above 36%	LEDs turn on then off without chatter	Observed: LEDs on at about 30 lux, still on at 36 lux, and off above 40 lux. Hysteresis confirmed.	Pass
T06 switch Manual; use vent and light buttons	Vent starts 0°, LEDs off; vent cycles 0°/45°/90°; light button toggles LEDs.	Observed: Manual entered at 0 deg/LEDs off; button cycled 45, 90, 0 deg; light button turned LEDs on. See T06 image.	Pass
T07 inject DHT fault while Manual	Fault overrides manual; N/A and CHECK DHT22	Observed: Injected DHT fault during Manual; SENSOR FAULT, N/A, CHECK DHT22, vent 45 deg, LEDs on. See T07 image.	Pass
T08 inject LDR fault in Auto	Fault; CHECK LDR, vent 45°, LEDs on	CHECK LDR; vent 45 deg; LEDs on. See T08 image.	Pass
T09 press reset with fault active or before 6 s	Fault remains latched	Observed: Pressing reset while DHT fault remained active left SENSOR FAULT latched. See T09 image.	Pass
T10 restore both inputs; wait 6 s; reset	Return to state selected by slide switch	Observed: After DHT fault off and over 6 s healthy, reset returned to Manual. See T10 image.	Pass
T11 send set hot 31 and set dark 40	Ack; outputs use new thresholds; UART streams	OK; 31 C vent 20 deg; 40% LEDs on; 49% off. See T11.	Pass

T12 enter malformed command and overflow buffer	ERR response; controller continues running	Observed: hot 50 ERR; long command ERR; status normal. See T12.	Pass
T13 disconnect actual DHT on physical rig	Fault at next DHT poll (~2 s); warning	Not run	Pending

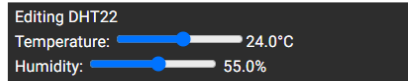
Photographic evidence

The following captures are arranged by test ID. T04 shows the vent closing as temperature drops below the 27 C threshold. T08 shows the injected LDR fault and safe output state.

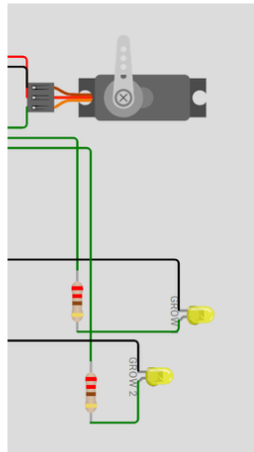
Test evidence | Wokwi simulation

T01 | Auto, 24 C, bright light: vent 0 degrees, LEDs off

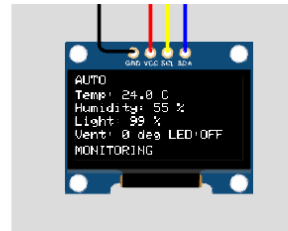
T01 | Auto, 24 C, bright light



Vent closed; LEDs off

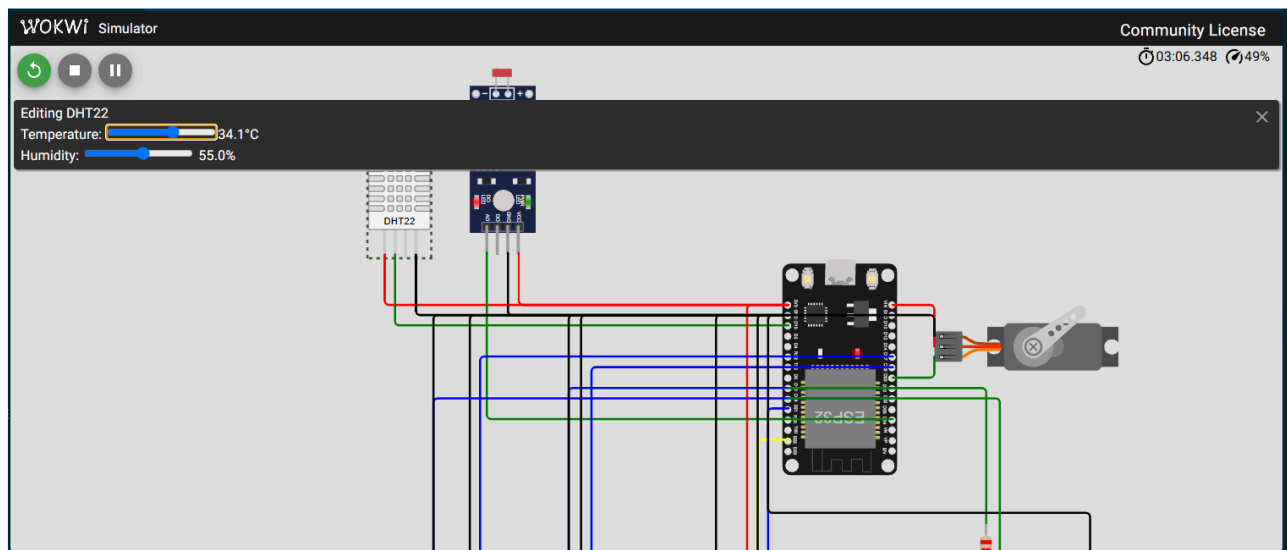


OLED: Auto, 24.0 C, light 99%



T02 | Warming: vent partly open

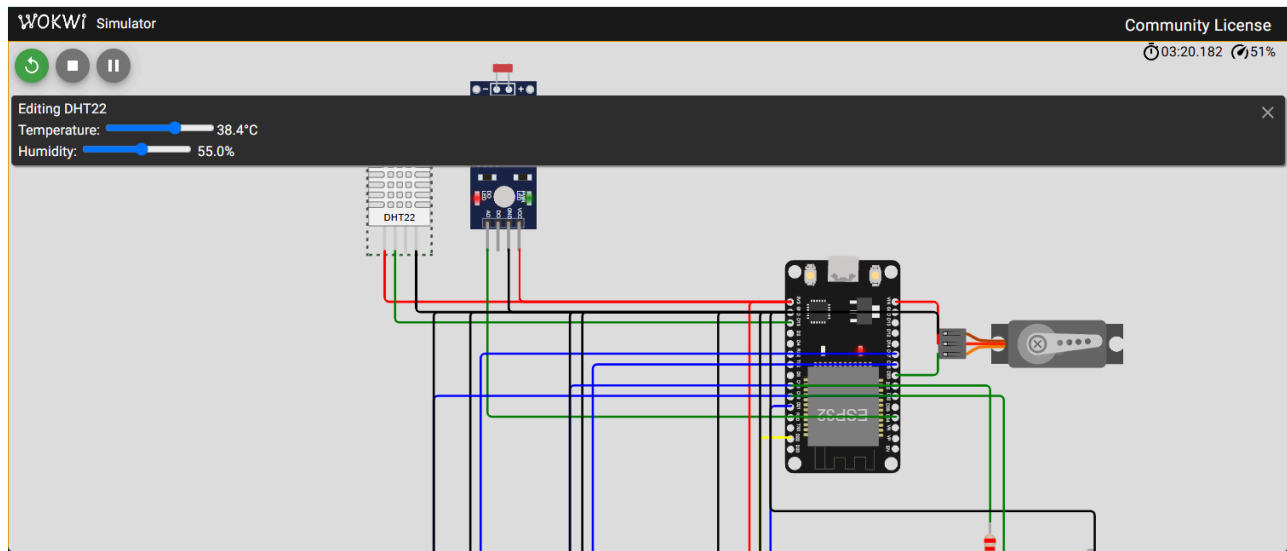
T02 | Auto, 34.1 C: vent partly open



Test evidence | Wokwi simulation

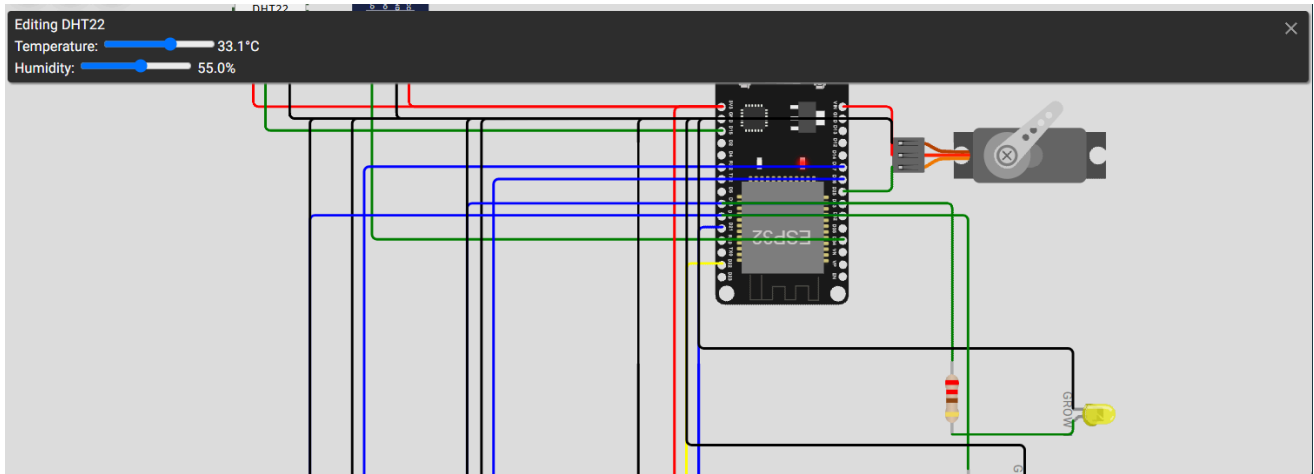
T03 | Hot condition: vent fully open

T03 | Auto, 38.4 C: vent fully open (90 deg)

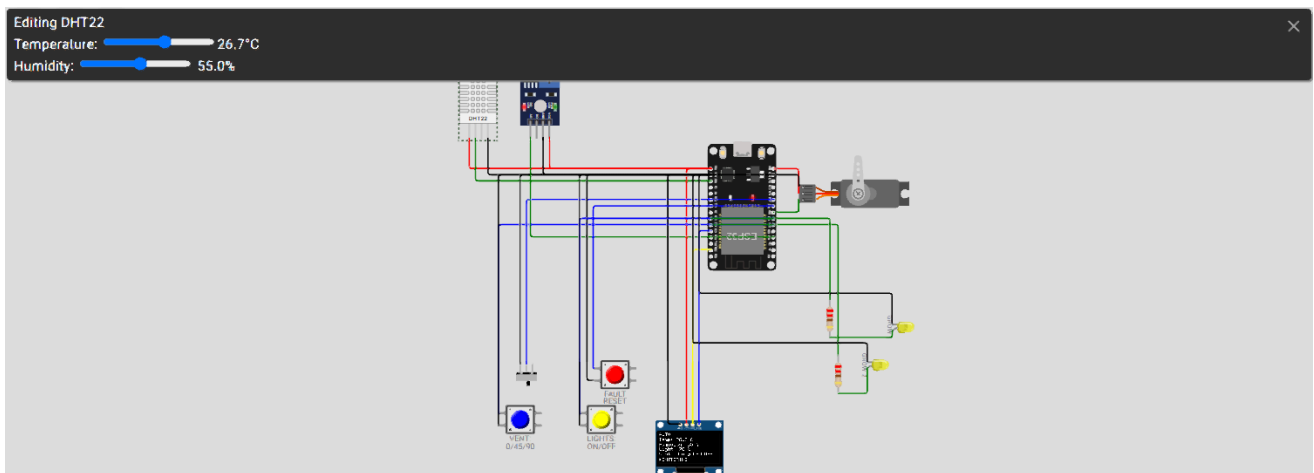


T04 | Auto cooling transition

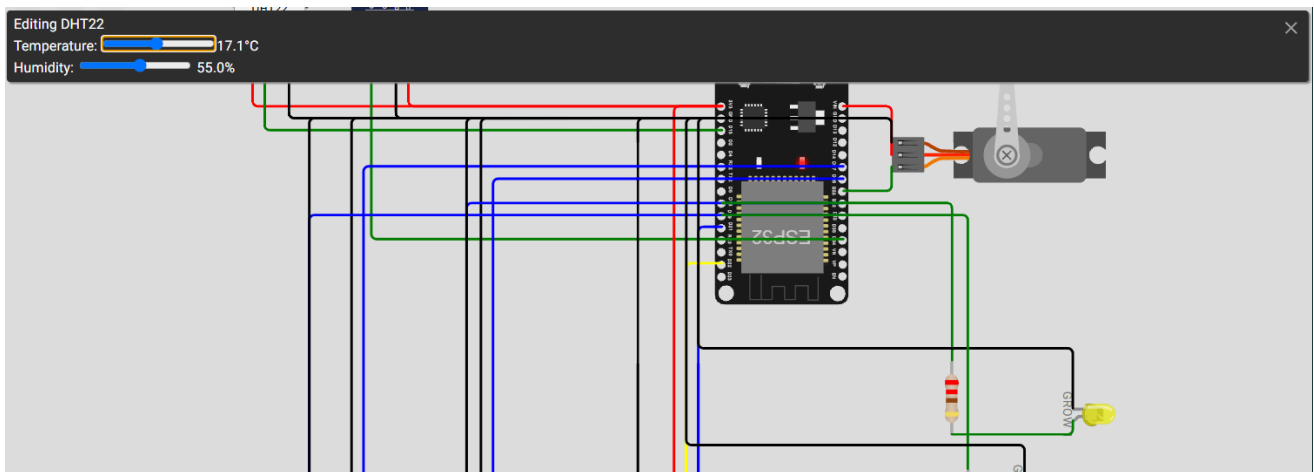
33.1 C | vent partly open



26.7 C | OLED reports vent 0 degrees



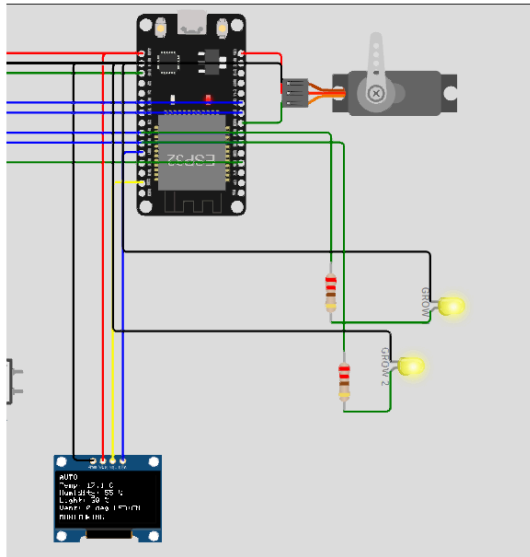
17.1 C | vent remains closed



Test evidence | Wokwi simulation

T05 | Dark end state: LEDs on

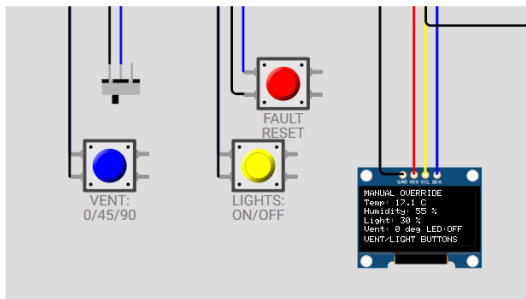
T04 / T05 | Auto, 17.1 C, light 30%



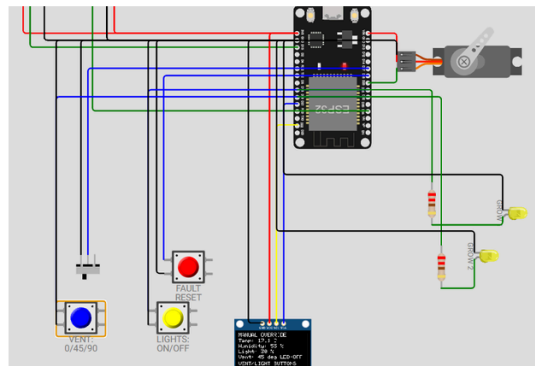
T06 | Manual vent and light override

T06 | Manual override: vent cycle and light toggle

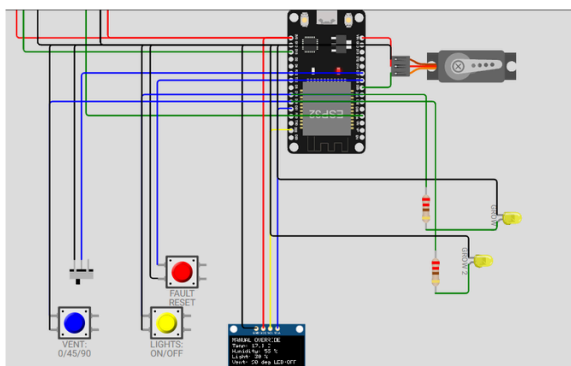
Manual entered: 0 deg, LEDs off



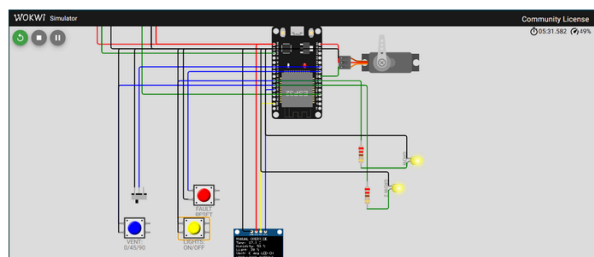
Vent button: 45 deg



Vent button: 90 deg



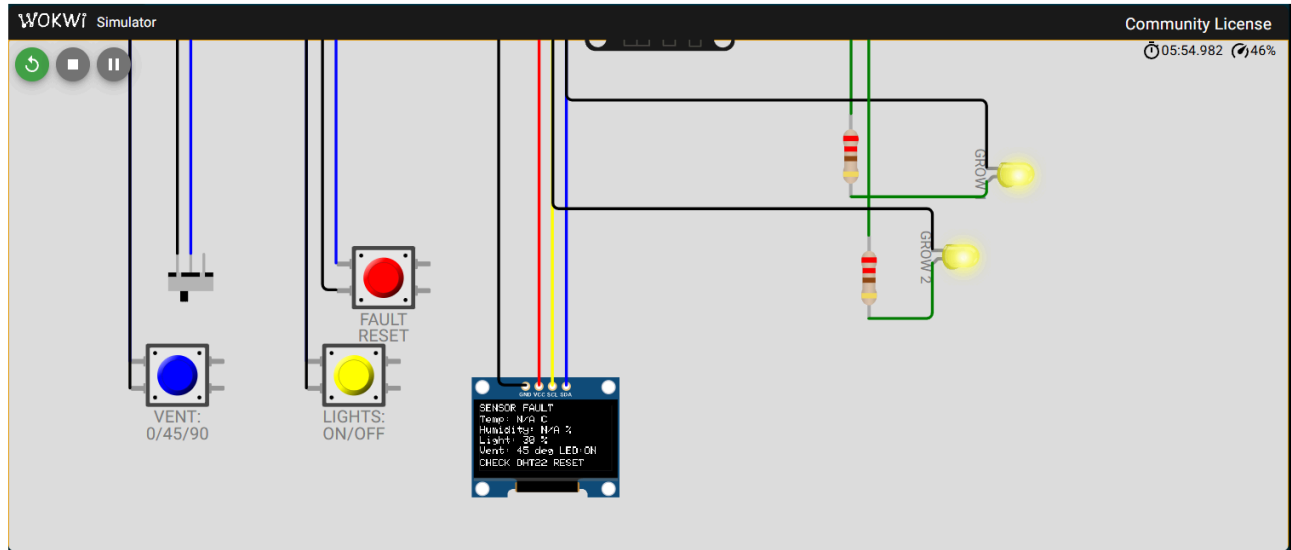
Light button: LEDs on



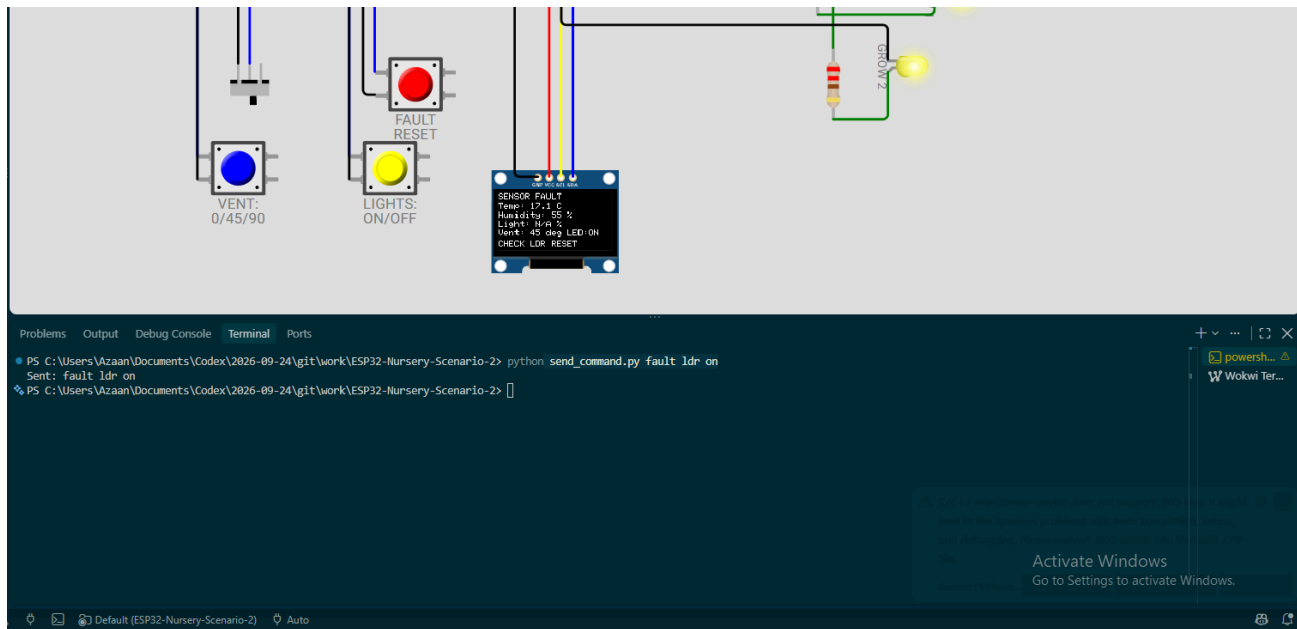
Test evidence | Wokwi simulation

T07 | DHT fault takes priority during Manual

T07 | DHT fault overrides Manual

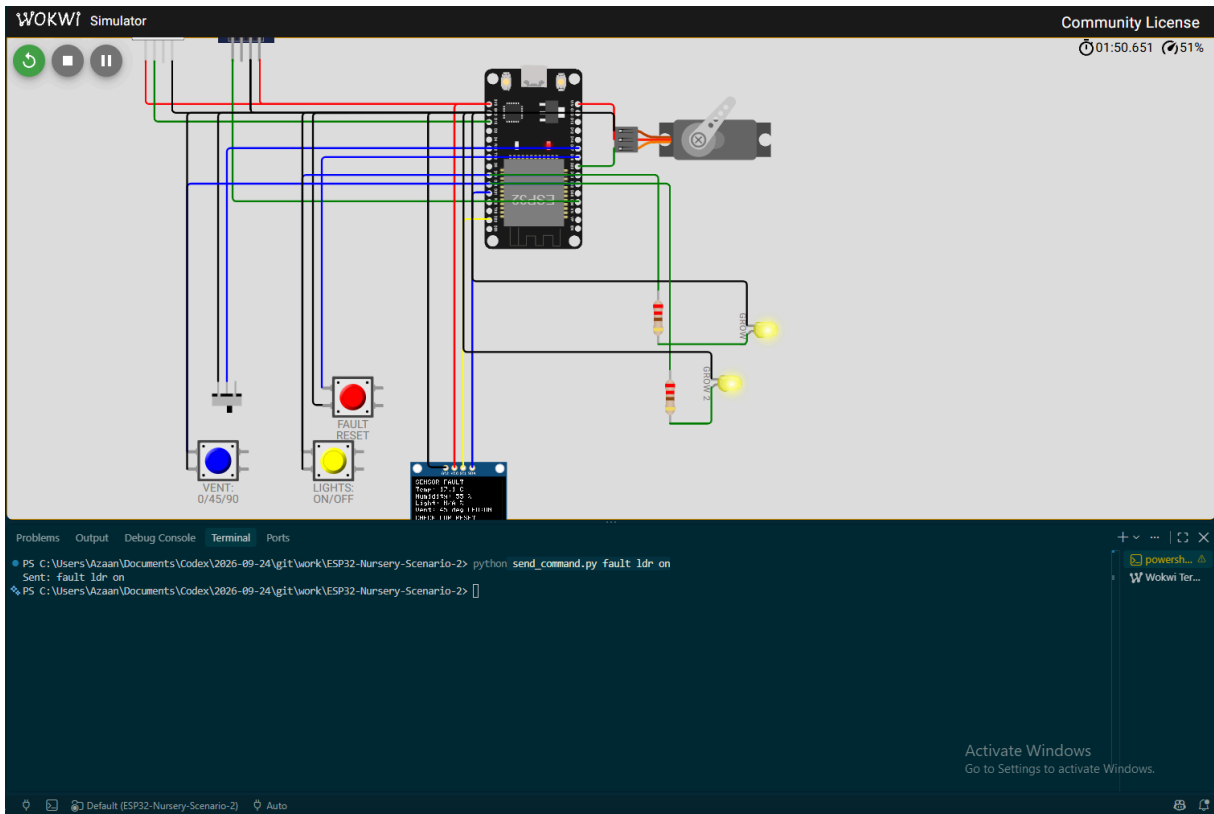


T08 | LDR fault: CHECK LDR, vent 45 degrees, LEDs on



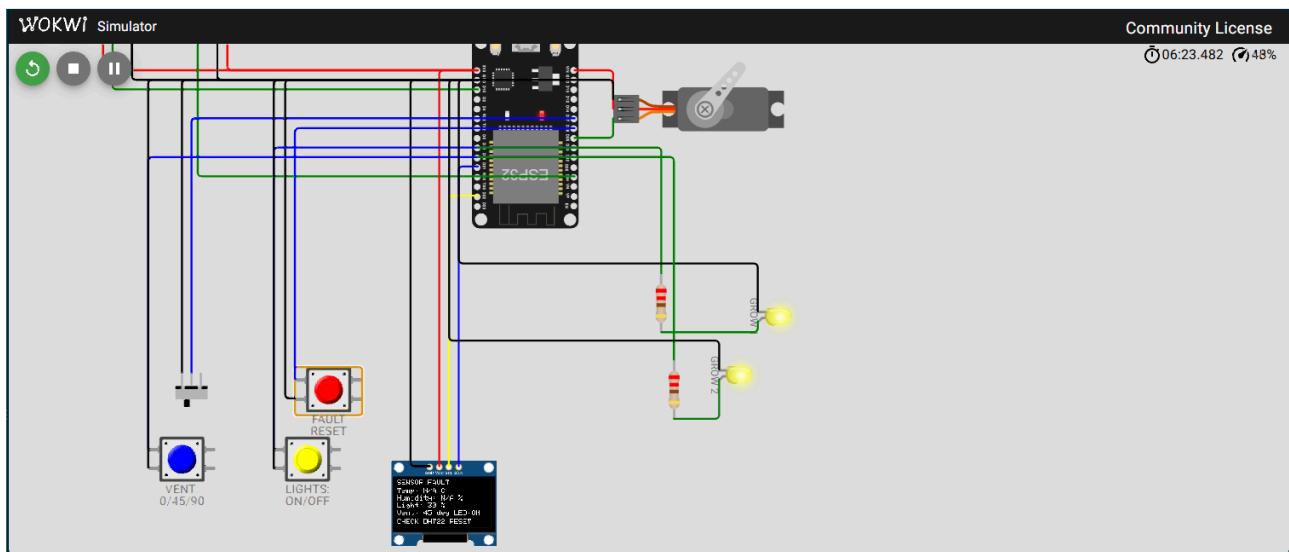
Test evidence | Wokwi simulation

T08 | LDR fault: full output view



T09 | Reset pressed while fault is active

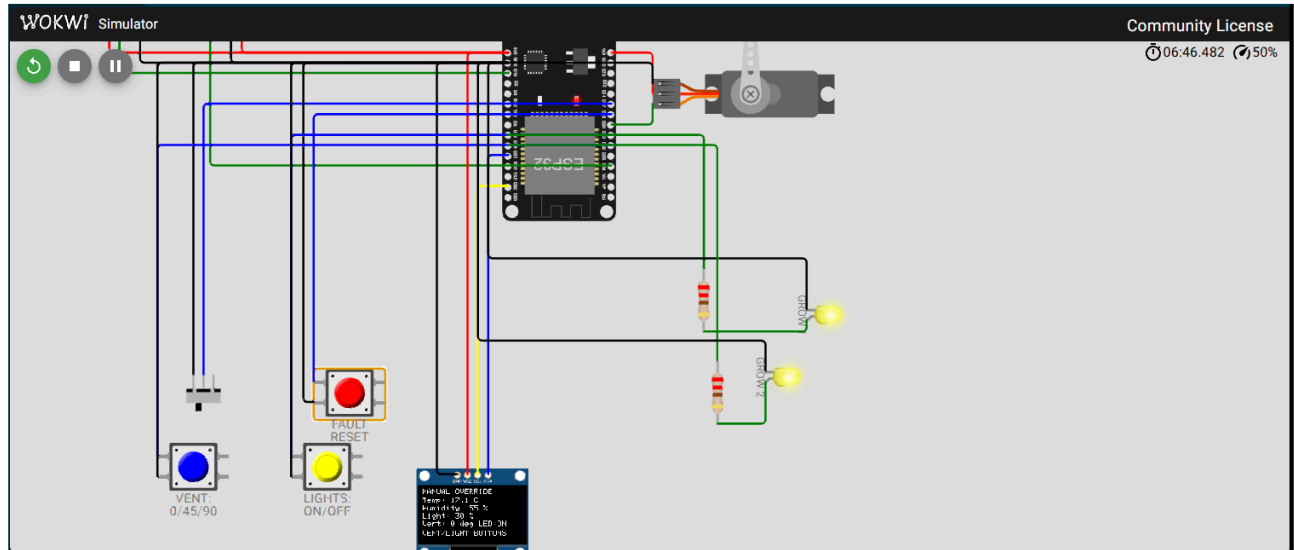
T09 | Reset while fault active: still latched



Test evidence | Wokwi simulation

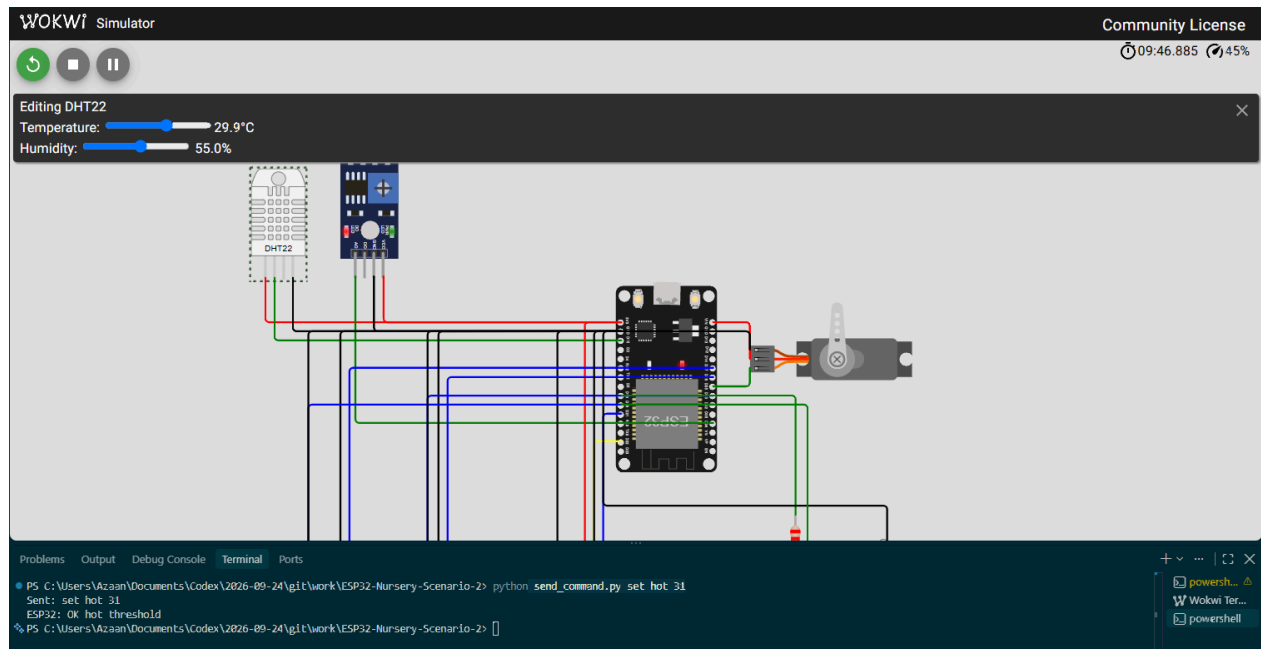
T10 | Recovery after sensor restored and reset

T10 | Fault cleared and reset: returns to Manual

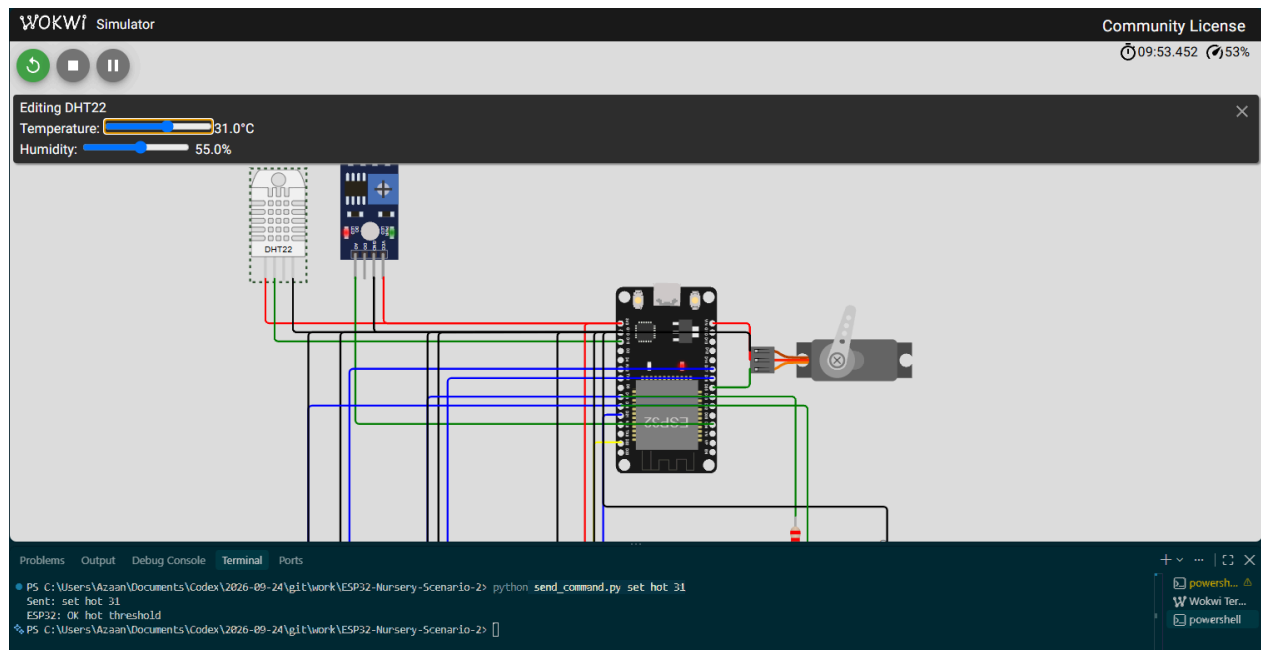


T11 | Serial threshold settings

set hot 31 | ESP32 acknowledges threshold



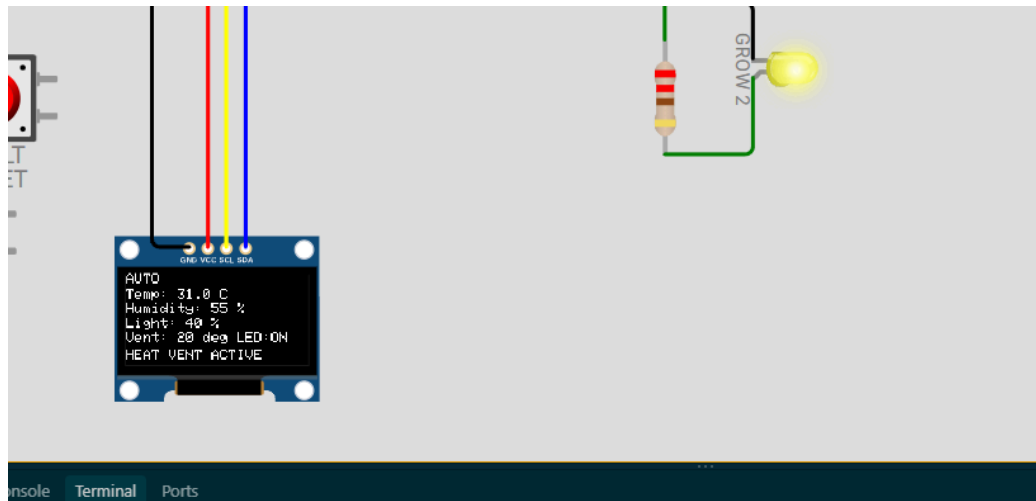
31.0 C | vent begins opening



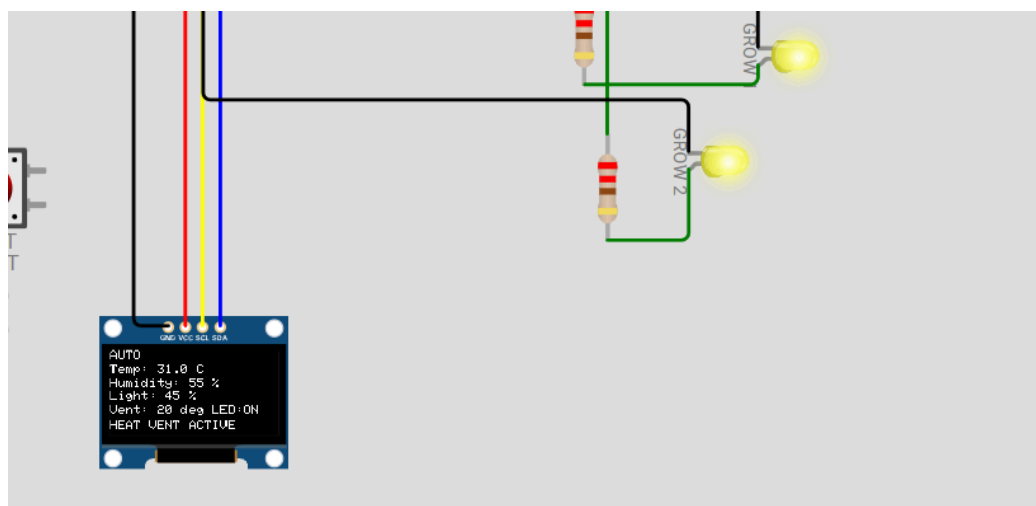
T11 | New light threshold and hysteresis

set dark 40 was acknowledged. The OLED's relative light value is used for the LED rule.

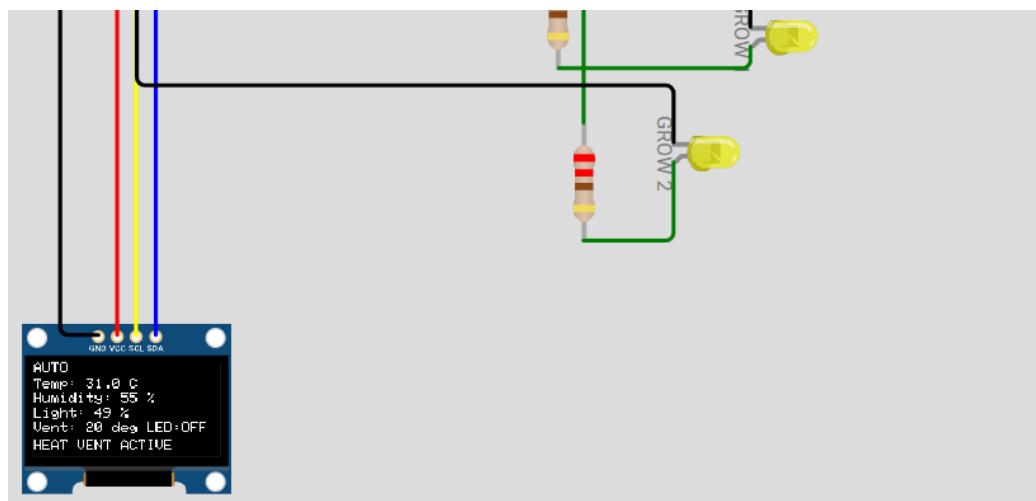
40% light | LEDs ON



45% light | LEDs remain ON



49% light | LEDs OFF



Test evidence | Wokwi simulation

T12 | Invalid value and long command rejected; status still works

```

Problems Output Debug Console Terminal Ports
PS C:\Users\Azaan\Documents\Codex\2026-09-24\git\work\ESP32-Nursery-Scenario-2> python send_command.py set dark 40
ESP32: OK dark threshold
● PS C:\Users\Azaan\Documents\Codex\2026-09-24\git\work\ESP32-Nursery-Scenario-2> python send_command.py set hot 50
Sent: set hot 50
ESP32: ERR hot must be 25..34 C
● PS C:\Users\Azaan\Documents\Codex\2026-09-24\git\work\ESP32-Nursery-Scenario-2> python send_command.py status
Sent: status
ESP32: mode=AUTO temp=31.0C humidity=55.0% light=49% raw=2078 vent=20deg leds=off dht=ok ldr=ok
PS C:\Users\Azaan\Documents\Codex\2026-09-24\git\work\ESP32-Nursery-Scenario-2> $longCommand = 'x' * 64
● >> python send_command.py $longCommand
Sent: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
ESP32: ERR command too long
● PS C:\Users\Azaan\Documents\Codex\2026-09-24\git\work\ESP32-Nursery-Scenario-2> python send_command.py status
Sent: status
ESP32: mode=AUTO temp=31.0C humidity=55.0% light=49% raw=2078 vent=20deg leds=off dht=ok ldr=ok
❖ PS C:\Users\Azaan\Documents\Codex\2026-09-24\git\work\ESP32-Nursery-Scenario-2> █

🔌 Default (ESP32-Nursery-Scenario-2) 🔌 Auto
```

Demonstration and recording notes

Before a lecturer demonstration, confirm live readings, LED response, proportional servo movement, Manual vent and light buttons, fault injection, protected reset and serial configuration in that order. Demonstrate a physical sensor fault if it can be done safely. Actual latency may exceed the nominal sampling period because library calls and display transfer take time; measure it with the serial log if timing matters.

For each test, record the starting mode, change only the named input, allow at least one sampling interval, and note the exact OLED message, servo angle and UART line. For the six-second recovery test, start timing only after both sensors are valid. A reset at five seconds should be rejected; a press after six seconds should succeed. Approach light thresholds from both directions to show hysteresis. For simultaneous conditions, enable Manual and inject a DHT fault nearly together; FAULT should take priority regardless of input order.

UART fault injection checks mode selection and recovery repeatably. Physically unplugging the DHT checks whether an actual communication failure yields an invalid sample. An LDR disconnected from GPIO34 may float at a plausible value, so one sensor cannot reliably detect every disconnection. Record the raw value if this occurs. An external pull-down, continuity circuit or second light sensor would change the physical fault model.

Repeat T01 after recovery and a power cycle to check normal outputs and whether unsaved UART setpoints return to defaults. Record dates, simulator or board version and relevant library versions. If an observed result differs from the expected result, record the discrepancy, likely cause and corrective action instead of marking the test Pass.

References

[1] Bluelab Support, "Meet your IntelliClimate."

<https://support.bluelab.com/intelliclimate-meet-your-intelliclimate> (accessed 24 September 2026).

[2] Priva, "Priva Connex: the most advanced greenhouse climate computer."

<https://www.priva.com/horticulture/solutions/climate-and-process-computers/priva-connex> (accessed 24 September 2026).

[3] Priva, "Tip: Optimizing greenhouse climate with vent control in Priva Connex and Compact CC," 9

September 2024.

<https://www.priva.com/blog/horticulture/tip-optimizing-greenhouse-climate-with-vent-control-in-priva-connex-and-compact-cc> (accessed 24 September 2026).

[4] Wokwi Docs, "DHT22 reference," "Photoresistor sensor reference," "SSD1306 board reference," and

"Diagram JSON file format." <https://docs.wokwi.com/parts/wokwi-dht22> ;

<https://docs.wokwi.com/parts/wokwi-photoresistor-sensor> ; <https://docs.wokwi.com/parts/board-ssd1306> ;

<https://docs.wokwi.com/diagram-format> (accessed 24 September 2026).

[5] APIIT Sri Lanka, COMP50069 Hardware Microcontrollers and Sensors assignment brief, supplied course document.

Appendix Full Doxygen Documented Source Code

The listing below reproduces the complete supplied sketch. The project root also contains diagram.json, libraries.txt and README.md.

```
/**
 * @file sketch.ino
 * @brief ESP32 nursery controller with Auto, Manual, and Sensor Fault modes.
 */
#include <Arduino.h>
#include <Wire.h>
#include <DHTesp.h>
#include <ESP32Servo.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <math.h>
#include <stdlib.h>
#include <string.h>

// Wiring
const byte DHT_PIN = 15, LDR_PIN = 34, SERVO_PIN = 25;
const byte LED1_PIN = 18, LED2_PIN = 19;
const byte MANUAL_PIN = 27, RESET_PIN = 26;
const byte VENT_BUTTON_PIN = 32, LIGHT_BUTTON_PIN = 33;

// Timing and safety settings
const unsigned long DHT_INTERVAL = 2000;
const unsigned long LIGHT_INTERVAL = 250;
const unsigned long SCREEN_INTERVAL = 250;
const unsigned long STATUS_INTERVAL = 1000;
const unsigned long RECOVERY_TIME = 6000;
const unsigned long DEBOUNCE_TIME = 40;
const int FAULT_VENT = 45;

enum Mode { AUTO, MANUAL, FAULT };
Mode mode = AUTO;
DHTesp dht;
Servo vent;
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
bool oledReady = false;

float temperatureC = NAN, humidityPct = NAN, hotThreshold = 29.0f;
int lightRaw = -1, lightPct = -1, darkThreshold = 30;
int ventDegrees = 0, manualVent = 0;
bool dhtValid = false, ldrValid = false, haveDhtSample = false;
bool injectDht = false, injectLdr = false;
bool hotDemand = false, autoLights = false, growLights = false;
bool manualOn = false, manualCandidate = false, manualLights = false;
bool wasHealthy = false;
volatile bool resetInterrupt = false;
unsigned long manualChanged = 0, healthySince = 0;
unsigned long lastDht = 0, lastLight = 0, lastScreen = 0, lastStatus = 0;

// Each button connects its GPIO pin to ground when pressed.
struct Button {
    byte pin;
    bool candidate;
    bool stable;
    unsigned long changed;
};
Button resetButton = {RESET_PIN, false, false, 0};
Button ventButton = {VENT_BUTTON_PIN, false, false, 0};
Button lightButton = {LIGHT_BUTTON_PIN, false, false, 0};

char command[64];
byte commandLength = 0;

/**
 * @brief Record that the reset button produced a falling edge.
 * @param none No arguments.
 * @return Nothing; the main loop checks the flag after debouncing.
 */
void IRAM_ATTR onResetPress() {
    resetInterrupt = true;
}

/**
 * @brief Return true once when a button has been held for 40 ms.
 * @param button Button and its remembered state.
 * @param now Current elapsed time in milliseconds.
```

```

* @return True once for each stable press.
*/
bool newPress(Button &button, unsigned long now) {
    bool pressed = digitalRead(button.pin) == LOW;
    if (pressed != button.candidate) {
        button.candidate = pressed;
        button.changed = now;
    }
    if (pressed != button.stable && now - button.changed >= DEBOUNCE_TIME) {
        button.stable = pressed;
        return pressed;
    }
    return false;
}

/**
 * @brief Read the Auto/Manual slide switch without contact bounce.
 * @param now Current elapsed time in milliseconds.
 * @return Nothing; updates the Manual switch state.
 */
void readManualSwitch(unsigned long now) {
    bool reading = digitalRead(MANUAL_PIN) == LOW;
    if (reading != manualCandidate) {
        manualCandidate = reading;
        manualChanged = now;
    }
    if (manualOn != manualCandidate && now - manualChanged >= DEBOUNCE_TIME) {
        manualOn = manualCandidate;
        if (manualOn) {
            manualVent = 0;
            manualLights = false;
        }
    }
}

/**
 * @brief Read the light sensor and calculate a relative brightness percentage.
 * @param none No arguments.
 * @return Nothing; updates the light reading and its validity.
 */
void readLight() {
    lightRaw = analogRead(LDR_PIN);
    ldrValid = !injectLdr && lightRaw > 1 && lightRaw < 4094;
    // This module's voltage rises as the room gets darker.
    lightPct = ldrValid ? (int)round((4095 - lightRaw) * 100.0f / 4095.0f) : -1;
}

/**
 * @brief Read the DHT22 and reject missing or out-of-range measurements.
 * @param none No arguments.
 * @return Nothing; updates temperature, humidity, and validity.
 */
void readDht() {
    TempAndHumidity reading = dht.getTempAndHumidity();
    haveDhtSample = true;
    dhtValid = !injectDht && isfinite(reading.temperature) &&
        isfinite(reading.humidity) && reading.temperature >= -40 &&
        reading.temperature <= 80 && reading.humidity >= 0 &&
        reading.humidity <= 100;
    temperatureC = dhtValid ? reading.temperature : NAN;
    humidityPct = dhtValid ? reading.humidity : NAN;
}

/**
 * @brief Choose Fault first, then Manual, then Auto.
 * @param now Current elapsed time in milliseconds.
 * @param resetPressed True for a new debounced reset press.
 * @return Nothing; updates the operating mode.
 */
void chooseMode(unsigned long now, bool resetPressed) {
    if (!haveDhtSample) return;
    if (!dhtValid || !ldrValid) {
        mode = FAULT;
        wasHealthy = false;
        return;
    }
    if (!wasHealthy) {
        healthySince = now;
        wasHealthy = true;
    }
    if (mode == FAULT && (!resetPressed || now - healthySince < RECOVERY_TIME)) return;
    mode = manualOn ? MANUAL : AUTO;
}

```

```

/**
 * @brief Calculate the vent angle and LED state for the active mode.
 * @param none No arguments.
 * @return Nothing; updates output targets.
 */
void chooseOutputs() {
    // Remember Auto's light decision separately from the Manual button setting.
    if (ldrValid) {
        if (lightPct <= darkThreshold) autoLights = true;
        else if (lightPct >= darkThreshold + 6) autoLights = false;
    }
    if (dhtValid) {
        if (temperatureC <= 17 || temperatureC <= hotThreshold - 2) hotDemand = false;
        else if (temperatureC >= hotThreshold) hotDemand = true;
    }

    if (mode == FAULT || !haveDhtSample) {
        ventDegrees = FAULT_VENT;
        growLights = true;
    } else if (mode == MANUAL) {
        ventDegrees = manualVent;
        growLights = manualLights;
    } else {
        // At 38 C or above the vent is fully open.
        float fraction = (temperatureC - (hotThreshold - 2)) /
            (38.0f - (hotThreshold - 2));
        ventDegrees = hotDemand ? constrain((int)round(90 * fraction), 0, 90) : 0;
        growLights = autoLights;
    }
}

/**
 * @brief Send output changes to the servo and both LEDs.
 * @param none No arguments.
 * @return Nothing; updates hardware outputs only when needed.
 */
void applyOutputs() {
    static int previousVent = -1, previousLights = -1;
    if (ventDegrees != previousVent) {
        vent.write(ventDegrees);
        previousVent = ventDegrees;
    }
    if ((int)growLights != previousLights) {
        digitalWrite(LED1_PIN, growLights ? HIGH : LOW);
        digitalWrite(LED2_PIN, growLights ? HIGH : LOW);
        previousLights = growLights;
    }
}

/**
 * @brief Return a short name for the current operating mode.
 * @param none No arguments.
 * @return The name of Auto, Manual Override, or Sensor Fault.
 */
const char *modeName() {
    if (mode == FAULT) return "SENSOR FAULT";
    if (mode == MANUAL) return "MANUAL OVERRIDE";
    return "AUTO";
}

/**
 * @brief Show readings, outputs, and any fault on the OLED.
 * @param none No arguments.
 * @return Nothing; updates the display if it is available.
 */
void showScreen() {
    if (!oledReady) return;
    oled.clearDisplay();
    oled.setTextSize(1);
    oled.setTextColor(SSD1306_WHITE);
    oled.setCursor(0, 0); oled.print(modeName());
    oled.setCursor(0, 11); oled.print("Temp: ");
    if (dhtValid) oled.print(temperatureC, 1); else oled.print("N/A");
    oled.print(" C");
    oled.setCursor(0, 21); oled.print("Humidity: ");
    if (dhtValid) oled.print(humidityPct, 0); else oled.print("N/A");
    oled.print(" %");
    oled.setCursor(0, 31); oled.print("Light: ");
    if (ldrValid) oled.print(lightPct); else oled.print("N/A");
    oled.print(" %");
    oled.setCursor(0, 41); oled.print("Vent: "); oled.print(ventDegrees);
    oled.print(" deg LED:"); oled.print(growLights ? "ON" : "OFF");
    oled.setCursor(0, 53);
    if (mode == FAULT) {

```

```

oled.print("CHECK ");
if (!dhtValid) oled.print("DHT22 ");
if (!ldrValid) oled.print("LDR ");
oled.print("RESET");
} else if (mode == MANUAL) oled.print("VENT/LIGHT BUTTONS");
else oled.print(hotDemand ? "HEAT VENT ACTIVE" : "MONITORING");
oled.display();
}

/**
 * @brief Print a single status line to the serial monitor.
 * @param none No arguments.
 * @return Nothing; writes one UART line.
 */
void printStatus() {
  Serial.printf("mode=%s temp=%.1fC humidity=%.1f%% light=%d%% raw=%d "
    "vent=%ddeg leds=%s dht=%s ldr=%s\n", modeName(),
    temperatureC, humidityPct, lightPct, lightRaw, ventDegrees,
    growLights ? "on" : "off", dhtValid ? "ok" : "fault",
    ldrValid ? "ok" : "fault");
}

/**
 * @brief Parse a number and check its allowed range.
 * @param text Characters to parse.
 * @param minimum Lowest accepted value.
 * @param maximum Highest accepted value.
 * @param value Receives the parsed number.
 * @return True when the entire input is one finite in-range number.
 */
bool readNumber(const char *text, float minimum, float maximum, float &value) {
  char *end;
  value = strtod(text, &end);
  while (*end == ' ') end++;
  return end != text && *end == '\0' && isfinite(value) &&
    value >= minimum && value <= maximum;
}

/**
 * @brief Run one complete command from the serial monitor.
 * @param line Writable command text ending in a null character.
 * @return Nothing; prints a response over UART.
 */
void runCommand(char *line) {
  if (strcmp(line, "help") == 0) {
    Serial.println("status | set hot 25..34 | set dark 5..80 | fault dht on/off | fault ldr on/off | reset");
  } else if (strcmp(line, "status") == 0) {
    printStatus();
  } else if (strncmp(line, "set hot ", 8) == 0) {
    float value;
    if (readNumber(line + 8, 25, 34, value)) {
      hotThreshold = value;
      Serial.println("OK hot threshold");
    } else Serial.println("ERR hot must be 25..34 C");
  } else if (strncmp(line, "set dark ", 9) == 0) {
    float value;
    if (readNumber(line + 9, 5, 80, value)) {
      darkThreshold = (int)round(value);
      Serial.println("OK dark threshold");
    } else Serial.println("ERR dark must be 5..80 percent");
  } else if (strcmp(line, "fault dht on") == 0 || strcmp(line, "fault dht off") == 0) {
    injectDht = strcmp(line, "fault dht on") == 0;
    if (injectDht) dhtValid = false;
    Serial.println("OK DHT fault injection; use reset button after recovery");
  } else if (strcmp(line, "fault ldr on") == 0 || strcmp(line, "fault ldr off") == 0) {
    injectLdr = strcmp(line, "fault ldr on") == 0;
    if (injectLdr) ldrValid = false;
    Serial.println("OK LDR fault injection; use reset button after recovery");
  } else if (strcmp(line, "reset") == 0) {
    chooseMode(millis(), true);
    Serial.println(mode == FAULT ? "WAIT for healthy sensors 6s then reset" : "OK reset");
  } else Serial.println("ERR unknown command; type help");
}

/**
 * @brief Collect serial bytes until a newline completes a command.
 * @param none No arguments.
 * @return Nothing; may execute one or more complete commands.
 */
void readSerial() {
  while (Serial.available()) {
    char c = (char)Serial.read();
    if (c == '\r') continue;
    if (c == '\n') {

```

```

    command[commandLength] = '\0';
    if (commandLength) runCommand(command);
    commandLength = 0;
} else if (commandLength < sizeof(command) - 1) {
    command[commandLength++] = c;
} else {
    commandLength = 0;
    Serial.println("ERR command too long");
}
}
}

/**
 * @brief Set up the hardware and start the timers.
 * @param none No arguments.
 * @return Nothing; Arduino entry point.
 */
void setup() {
    Serial.begin(115200);
    pinMode(MANUAL_PIN, INPUT_PULLUP);
    pinMode(RESET_PIN, INPUT_PULLUP);
    pinMode(VENT_BUTTON_PIN, INPUT_PULLUP);
    pinMode(LIGHT_BUTTON_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(RESET_PIN), onResetPress, FALLING);
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);
    analogReadResolution(12);
    analogSetPinAttenuation(LDR_PIN, ADC_11db);
    Wire.begin(21, 22);
    oledReady = oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    dht.setup(DHT_PIN, DHTesp::DHT22);
    vent.setPeriodHertz(50);
    vent.attach(SERVO_PIN, 500, 2400);
    readLight();
    chooseOutputs();
    applyOutputs();
    showScreen();
    Serial.println("NURSERY READY; type help. Waiting for first DHT reading.");
    if (!oledReady) Serial.println("WARNING OLED missing at I2C address 0x3C");
    lastDht = lastLight = lastScreen = lastStatus = millis();
}

/**
 * @brief Read inputs, update the mode, and run each scheduled task.
 * @param none No arguments.
 * @return Nothing; Arduino entry point.
 */
void loop() {
    unsigned long now = millis();
    readSerial();
    readManualSwitch(now);
    bool resetPressed = newPress(resetButton, now) && resetInterrupt;
    if (resetPressed) resetInterrupt = false;
    bool ventPressed = newPress(ventButton, now);
    bool lightPressed = newPress(lightButton, now);

    if (now - lastLight >= LIGHT_INTERVAL) {
        lastLight = now;
        readLight();
    }
    if (now - lastDht >= DHT_INTERVAL) {
        lastDht = now;
        readDht();
    }
    chooseMode(now, resetPressed);
    if (mode == MANUAL) {
        if (ventPressed) manualVent = (manualVent + 45) % 135;
        if (lightPressed) manualLights = !manualLights;
    }
    chooseOutputs();
    applyOutputs();
    if (now - lastScreen >= SCREEN_INTERVAL) {
        lastScreen = now;
        showScreen();
    }
    if (now - lastStatus >= STATUS_INTERVAL) {
        lastStatus = now;
        printStatus();
    }
}

```